Griffith UNIVERSITY

3031CT: Applied Data Mining

3804ICT: Data Mining

Lecture 9:
Text Data Mining



Dr Can Wang (Gold Coast)

School of Information and Communication Technology

Text Data Mining

- Introduction to Text Data Mining
- Information Retrieval (IR)
  - IR architecture
  - IR Queries
    - Keyword queries
    - Boolean queries
    - Phrase queries
    - Proximity queries
    - Full document queries
- Information retrieval models
- Boolean model
- Vector space model
  - TF & TFIDF
  - Cosine similarity
- Statistical model
- Text Pre-processing
  - Stopword removal
  - Stemming
- Evaluations
  - Recall/Precision
  - Precision-Recall Curve
  - F-score

Mining Text Data: An Introduction

Griffith UNIVERSITY

Data Mining / Knowledge Discovery



Structured Data

HomeLoan (
Loanee: *Frank Rizzo*
Lender: *MWF*
Agency: *Lake View*
Amount: *$200,000*
Term: *15 years*
)

Multimedia



Loans(*$200K,[map],...*)

Free Text

Frank Rizzo bought his home from Lake View Real Estate in 1992.
He paid $200,000 under a15-year loan from MW Financial.

Hypertext

<a href>Frank Rizzo Bought <a hef>this home from <a href>Lake View Real Estate In 1992. <p>...

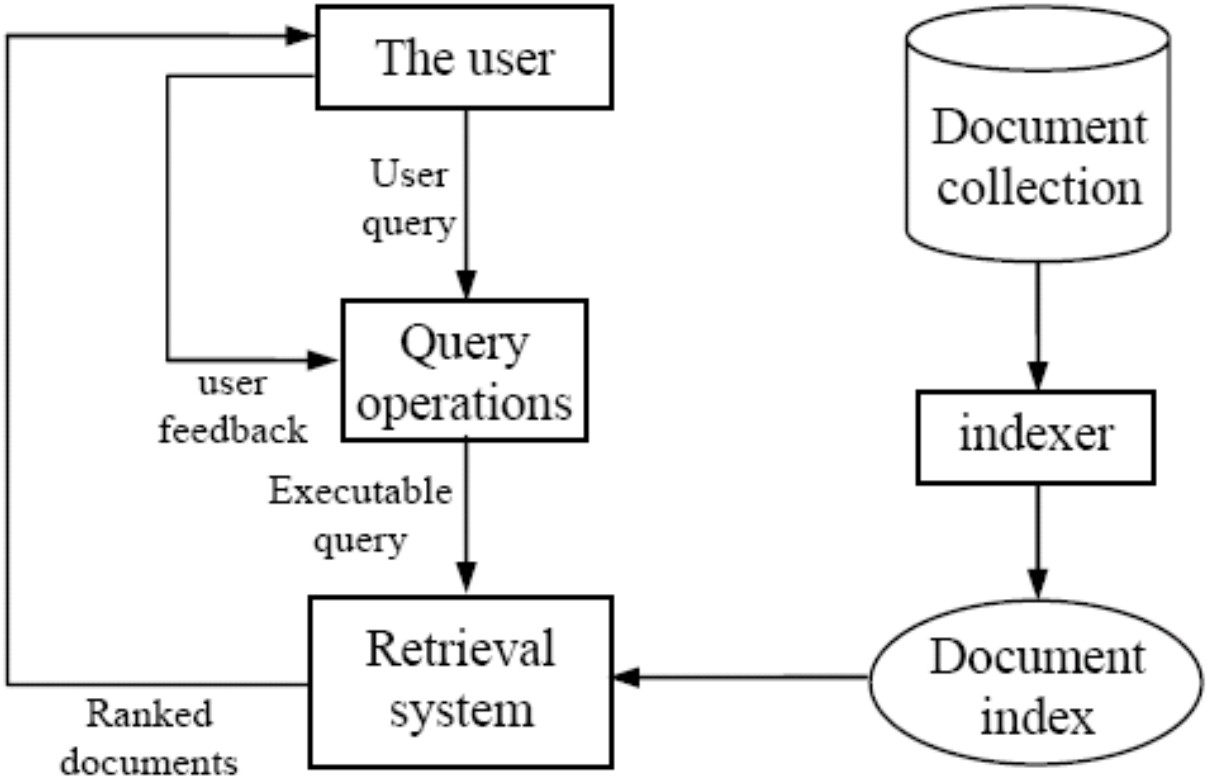
Introduction

- Text mining refers to data mining using text documents as data.
- Most text mining tasks use Information Retrieval (IR) methods to pre-process text documents.
- These methods are quite different from traditional data pre-processing methods used for relational tables.
- Web search also has its root in IR.

Information Retrieval (IR)

- Conceptually, IR is the study of finding needed information. IR helps users find information that matches their information needs.
- Expressed as queries
- Historically, IR is about document retrieval, emphasizing document as the basic unit.
- We need to find documents relevant to user queries
- Technically, IR studies the acquisition, organization, storage, retrieval, and distribution of information.

IR Architecture



IR Queries: Keyword Queries

Griffith UNIVERSITY

- **Keyword queries**
  - The user expresses his/her information needs with **a list of** (at least one) **keywords** (or **terms**) aiming to find documents that contain some (at least one) or all the query terms.
  - The **terms** in the list are assumed to be connected with a "soft" version of the logical AND.
  - The retrieval system then finds those likely **relevant documents** and **ranks** them suitably to present to the user.

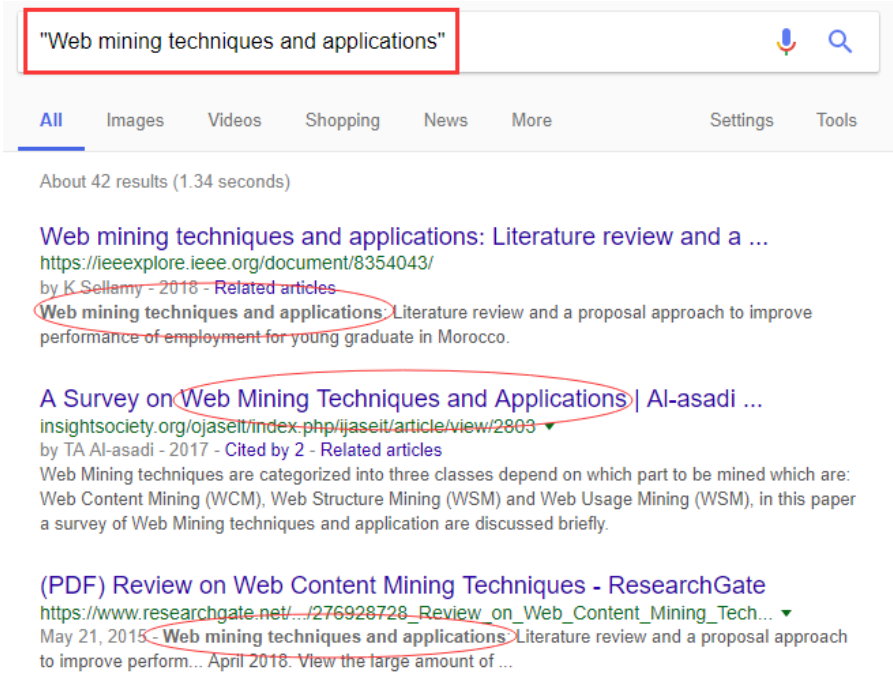
IR Queries: Boolean Queries

Griffith UNIVERSITY

- **Boolean queries** (using AND, OR, NOT)
  - The user can use Boolean operators, **AND**, **OR**, and **NOT** to construct complex queries.
  - A document is returned for a Boolean query if the query is logically true in the document (i.e., exact match).
  - Although one can write complex Boolean queries using the three operators, users seldomly write such queries.

IR Queries: Phrase Queries

- **Phrase queries**
  - A query consists of **a sequence of words** that makes up a phrase.
  - Each returned document must contain **at least one** instance of the phrase.
  - A phrase query is normally enclosed with **double quotes**.



IR Queries: Proximity Queries

Griffith UNIVERSITY

- **Proximity queries**
  - The proximity query is a **relaxed** version of the phrase query and can be a combination of terms and phrases.
  - Proximity queries seek the query terms within **close proximity** to each other.
  - The **closeness** is used as a factor in ranking the returned documents or pages.
  - Most search engines consider both **term proximity** and **term ordering** in retrieval.

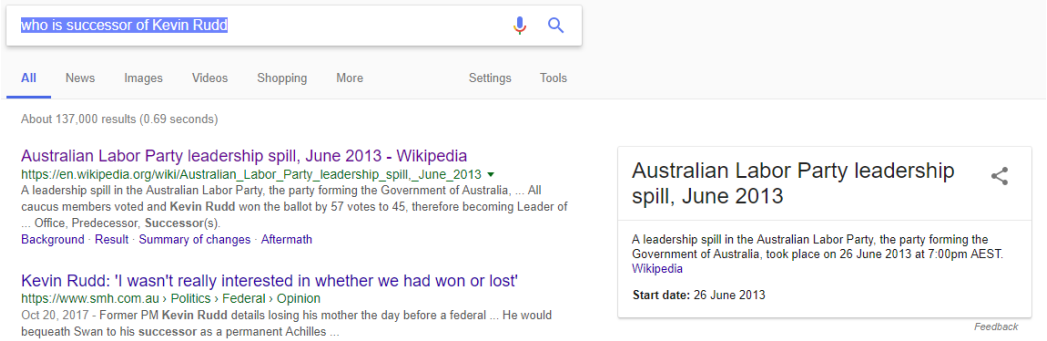
IR Queries: Full Document Queries

Griffith UNIVERSITY

- **Full document queries**
  - When the query is a full document, the user wants to find other documents that are similar to the query document.
  - Some search engines (e.g., Google) allow the user to issue such a query by providing the URL of a query page.
  - Google Cloud Document and Index https://cloud.google.com/appengine/docs/standard/python/search/

IR Queries: Natural Language Questions

- **Natural language questions**
  - The most complex case, but the ideal case.
  - The user expresses his/her information need as a natural language question.
  - The system then finds the answer.
  - Eg. “Who is successor of Kevin Rudd?” Is the answer “Julia Gillard” or “Tony Abbott”?





2007-2010;2013



2010-2013



2013-2015

  - Such queries are still hard to handle due to the difficulty of natural language understanding.
  - Nevertheless, this is an active research area, called **question answering**.

Information Retrieval Models

- An IR model governs how a document and a query are represented and how the relevance of a document to a user query is defined.
- Main models
  - Boolean model
  - Vector space model
  - Statistical language model
- Three models treat each document or query as a "**bag" of words** or **terms**. Term **sequence** and **position** in a sentence or a document are ignored.
- Given a collection of documents D, let $V = \{t_1, t_2, \ldots, t_{|V|}\}$ be the set of distinctive terms in the collection, where t_i is a term. The set V is usually called the vocabulary of the collection, and |V| is its size, i.e., the number of terms in V.
- A weight $w_{ij} > 0$ is associated with each term t_i of a document $d_j \in D$. For a term that does not appear in document d_j, $w_{ij} = 0$. Each document d_j is thus represented with a term vector:

$$\boldsymbol{d}_j = (w_{1j}, w_{2j}, \ldots, w_{|V|j}),$$

- Where each weight w_{ij} corresponds to the term $t_i \in V$, and quantifies the level of importance of t_i in document $\boldsymbol{d}_j$. The sequence of the components (or terms) in the vector **is not significant.**

Boolean Model

- The Boolean model is one of the earliest and simplest information retrieval models.
- It uses the notion of exact matching to match documents to the user query.
- Both the query and the retrieval are based on Boolean algebra: AND, OR, NOT.
- Documents and queries are represented as sets of terms.
- Each term is only considered present or absent in a document, by using the vector representation of the document, the weight of term t_i in document d_j is 1 if t_i appears in document d_j, and 0 otherwise, i.e.,

$$w_{ij} = \begin{cases} 1 & if\ t_i\ appears\ in\ d_j \\ 0 & otherwise. \end{cases}$$

Boolean Model

- Query terms are combined logically using the Boolean operators **AND**, **OR**, and **NOT.**
  - E.g., ((*data* AND *mining*) AND (NOT *text*))
- Retrieval
  - Given a Boolean query, the system retrieves every document that makes the query logically true.
  - Called **exact match**.
- But the retrieval results are usually quite poor because term frequency is not considered.

Vector Space Model

- Each document is represented as a vector.
- However, the term weights are no longer 0 or 1. Each term weight is computed based on some variations of TF or TF-IDF scheme.
- Term Frequency (TF) Scheme: The weight of a term t_i in document $\mathbf{d}_j$ is the number of times that t_i appears in $\mathbf{d}_j$, denoted by f_{ij}. Normalization may also be applied.

TF(t_i) = (Number of times term t_i appears in a document $\mathbf{d}_j$) / (Total number of terms in the document $\mathbf{d}_j$).

- The shortcoming of the TF scheme is that it does not consider the situation where a term appears in many documents of the collection. Such a term may not be discriminative.

TF-IDF term weighting scheme

- The most well-known weighting scheme
  - TF: still **term frequency**
  - IDF: **inverse document frequency**

  N: total number of docs

  df_i: the number of docs that t_i appears.

$$idf_i = \log \frac{N}{df_i}$$

- The final TF-IDF term weight is:

$$w_{ij} = tf_{ij} \times idf_i.$$

IDF example, suppose N = 1 million

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
Fly	10,000	2
under	100,000	1
the	1,000,000	0

$$\text{idf}_t = \log_{10}(N/\text{df}_t)$$

There is one idf value for each term *t* in a collection

Use log is to "dampen" the effect of idf

Retrieval in vector space model

- Query **q** is represented in the same way or slightly differently.
- Relevance of $\mathbf{d}_j$ to **q**: Compare the similarity of query **q** and document $\mathbf{d}_j$.
- Cosine similarity (the cosine of the angle between the two vectors)

$$cosine(\mathbf{d}_j, \mathbf{q}) = \frac{\langle \mathbf{d}_j \bullet \mathbf{q} \rangle}{\| \mathbf{d}_j \| \times \| \mathbf{q} \|} = \frac{\sum_{i=1}^{|V|} w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} \times \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

- Cosine is also commonly used in text clustering

Statistical Model

- Statistical language models (or simply **language models**) are based on probability and have foundations in statistical theory.
- The basic idea of this approach is to estimate the probabilities of a query generated by documents: the probability of a query q is "generated" by a probabilistic model based on a document d_j, i.e., $\Pr(q|d_j)$
- Rank documents by the likelihood of the query given the language model.
- Similar ideas have previously been used in natural language processing and speech recognition.

Text Pre-processing

- Word (term) extraction: easy
- Stopword removal
- Stemming
- Frequency counts and computing TF-IDF term weights



Stopword Removal

- Many of the most frequently used words in English are useless in IR and text mining – these words are called *stopwords*.
  - the, of, and, to, be, in, on, ….
  - Typically about 400 to 500 such words
  - For an application, an additional domain specific stopwords list may be constructed
- Why do we need to remove stopwords?
  - Reduce indexing (or data) file size
    - stopwords accounts 20-30% of total word counts
  - Improve efficiency and effectiveness
    - stopwords are not useful for searching or text mining
    - they may also confuse the retrieval system

Stemming

- Techniques used to find out the root/stem of a word. E.g.,
  - user — engineering
  - users — engineered
  - used — engineer
  - using
- stem: use — engineer

Usefulness:

- improving effectiveness of IR and text mining
  - matching similar words
  - Mainly improving recall
- reducing indexing size
  - combining words with the same roots may reduce indexing size as much as 40-50%

Basic Stemming Methods

Griffith UNIVERSITY

Using a set of rules. E.g.,

- Remove ending
  - If a word ends with a consonant other than s, followed by an s, then delete s.
  - If a word ends in es, drop the s.
  - If a word ends in ing, delete the ing unless the remaining word consists only of one letter, or of th.
  - If a word ends with ed, preceded by a consonant, delete the ed unless this leaves only a single letter.
  - ……
- Transform words
  - If a word ends with “ies” but not “eies” or “aies” then “ies --> y.”

Frequency counts + TF-IDF

- Count the number of times a word occurred in a document.
  - Use occurrence frequencies to indicate relative importance of a word in a document.
    - ❑ If a word appears often in a document, the document likely "deals with" subjects related to the word.
- Count the number of documents in the collection that contains each word
- TF-IDF can be computed.

$$tf_{ij} = \frac{f_{ij}}{\max\{f_{1j}, f_{2j}, \ldots, f_{|V|j}\}}$$

$$idf_i = \log \frac{N}{df_i}$$

$$w_{ij} = tf_{ij} \times idf_i$$

Evaluation: Precision and Recall

Griffith UNIVERSITY

- Given a query:
  - Are all retrieved documents relevant?
  - Have all the relevant documents been retrieved?
- Measures for system performance:
  - The first question is about the precision of the search
  - The second is about the completeness (recall) of the search.

Evaluation: Precision and Recall

- Let the collection of documents in the database be D, and the total number of documents in D be N. Given a user query q, the retrieval algorithm first computes the relevance scores for all documents in D, and then produce a ranking R_q of the documents based on the relevance scores: $R_q: < d_1^q, d_2^q, \ldots, d_N^q >$, where d_1^q is the most relevant document to query q and d_N^q is the most irrelevant document to query q.

- Let D_q be the set of actual relevant documents of query **q** in *D*.

Evaluation: Precision and Recall

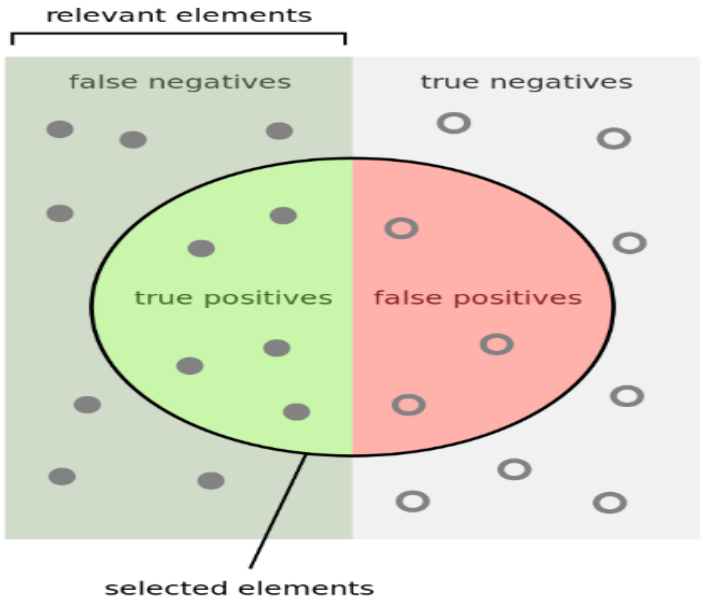
- We compute the precision and recall values at each d_i^q in the ranking.
- **Recall** at rank position *i*

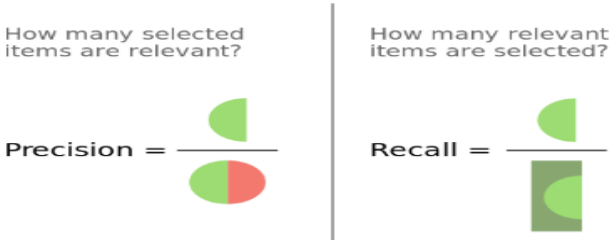
$$r(i) = \frac{s_i}{|D_q|}.$$

- **Precision** at rank position *i*

$$p(i) = \frac{s_i}{i}$$

s_i: the number of **relevant** documents from d_1^q to d_i^q in R_q





Example: Precision and Recall

- We have a document collection *D* with 20 documents. Given a query **q**, we know that eight documents are relevant to **q**. A retrieval algorithm produces the ranking (of all documents in *D*).
- In column 1 of the table, 1 represents the highest rank and 20 represents the lowest rank.
- In column 2 of the table, "+" and "-" indicate a relevant document and an irrelevant document respectively.
- The precision ($p(i)$) and recall ($r(i)$) values at each position i are given in columns 3 and 4.

$$p(i) = \frac{s_i}{i} \qquad r(i) = \frac{s_i}{|D_q|}.$$

Rank *i*	+/–	*p*(*i*)	*r*(*i*)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	–	7/11 = 63%	7/8 = 88%
12	–	7/12 = 58%	7/8 = 88%
13	+	8/13 = 62%	8/8 = 100%
14	–	8/14 = 57%	8/8 = 100%
15	–	8/15 = 53%	8/8 = 100%
16	–	8/16 = 50%	8/8 = 100%
17	–	8/17 = 47%	8/8 = 100%
18	–	8/18 = 44%	8/8 = 100%
19	–	8/19 = 42%	8/8 = 100%
20	–	8/20 = 40%	8/8 = 100%

Average Precision

- Sometimes we want a single precision to compare different retrieval algorithms on a query **q**. **An average precision (*p*_avg)** can be computed based on the precision at each relevant document in the ranking

$$p_{avg} = \frac{\sum_{d_i^q \in D_q} p(i)}{|D_q|}.$$

$$p_{avg} = \frac{100\% + 100\% + 100\% + 80\% + 71\% + 67\% + 70\% + 62\%}{8} = 81\%.$$

Rank *i*	+/–	*p*(*i*)	*r*(*i*)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	–	7/11 = 63%	7/8 = 88%
12	–	7/12 = 58%	7/8 = 88%
13	+	8/13 = 62%	8/8 = 100%
14	–	8/14 = 57%	8/8 = 100%
15	–	8/15 = 53%	8/8 = 100%
16	–	8/16 = 50%	8/8 = 100%
17	–	8/17 = 47%	8/8 = 100%
18	–	8/18 = 44%	8/8 = 100%
19	–	8/19 = 42%	8/8 = 100%
20	–	8/20 = 40%	8/8 = 100%

Precision-Recall Curve

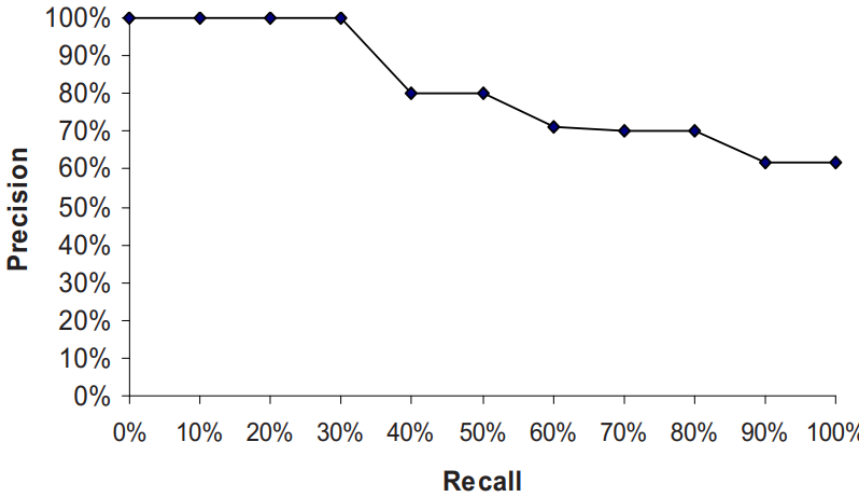
- Based on the precision and recall values at each rank position, we can draw a precision–recall curve
- the *x*-axis is the recall and the *y*-axis is the precision.
- Instead of using the precision and recall at each rank position, the curve is commonly plotted using 11 standard recall levels, 0%, 10%, 20%, …, 100%.
- Since we may not obtain exactly these recall levels in the ranking, **interpolation** is needed to obtain the precisions at these recall levels, which is done as follows: Let r_i be a recall level, $i \in \{0,1,2,\ldots,10\}$, and $p(r_i)$ be the precision at the recall level r_i .
- $p(r_i)$ is computed with

$$p(r_i) = \max_{r_i \le r \le r_{10}} p(r)$$

Precision-Recall Curve

- The precision-recall curve of the example is

i	$p(r_i)$	r_i
0	100%	0%
1	100%	10%
2	100%	20%
3	100%	30%
4	80%	40%
5	80%	50%
6	71%	60%
7	70%	70%
8	70%	80%
9	62%	90%
10	62%	100%



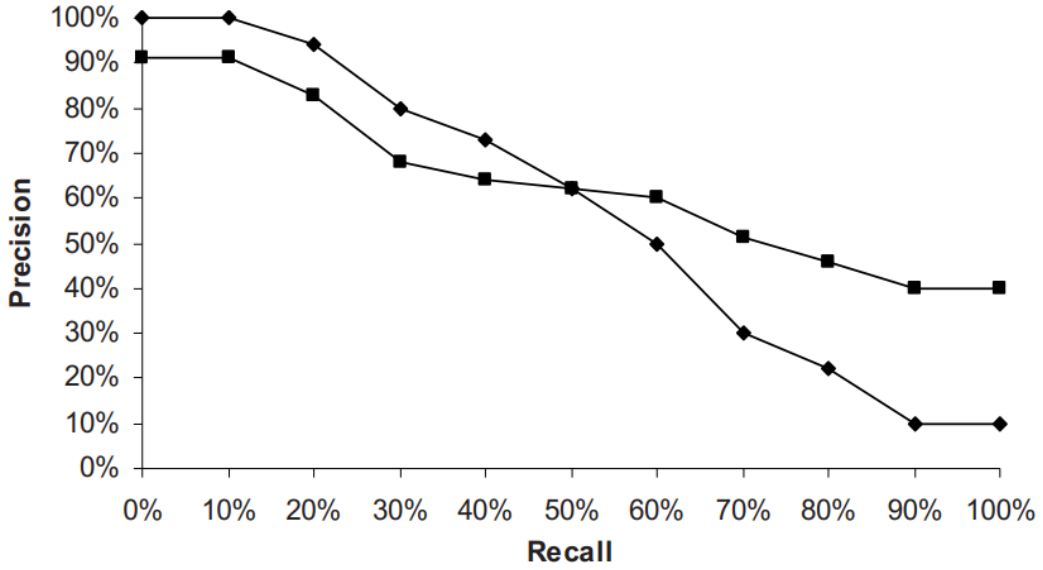
$$p(r_i) = \max_{r_i \le r \le r_{10}} p(r)$$

Rank i	+/–	$p(i)$	$r(i)$
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	–	7/11 = 63%	7/8 = 88%
12	–	7/12 = 58%	7/8 = 88%
13	+	8/13 = 62%	8/8 = 100%
14	–	8/14 = 57%	8/8 = 100%
15	–	8/15 = 53%	8/8 = 100%
16	–	8/16 = 50%	8/8 = 100%
17	–	8/17 = 47%	8/8 = 100%
18	–	8/18 = 44%	8/8 = 100%
19	–	8/19 = 42%	8/8 = 100%
20	–	8/20 = 40%	8/8 = 100%

Compare different retrieval algorithms

Griffith UNIVERSITY

- Frequently, we need to compare the retrieval results of different algorithms. We can draw their precision-recall curves together in the same figure for comparison.
- Precisions of one algorithm are better than those of the other at low recall levels, but are worse at high recall levels



F-score

- Another often used evaluation measure is F-score
- Here we can compute the F-score at each rank position *i:*

$$F(i) = \frac{2}{\frac{1}{r(i)} + \frac{1}{p(i)}} = \frac{2\,p(i)r(i)}{p(i) + r(i)}.$$

Text Data Mining

- Introduction to Text Data Mining
- Information Retrieval (IR)
  - IR architecture
  - IR Queries
    - Keyword queries
    - Boolean queries
    - Phrase queries
    - Proximity queries
    - Full document queries
- Information retrieval models
- Boolean model
- Vector space model
  - TF & TFIDF
  - Cosine similarity
- Statistical model
- Text Pre-processing
  - Stopword removal
  - Stemming
- Evaluations
  - Recall/Precision
  - Precision-Recall Curve
  - F-score

35

Have a question?

